

Hochschule Bremerhaven

Bremerhaven University of Applied Sciences

Master of Science in Embedded System Design WS25

Digital Systems and VHDL 3,4

1 Mohammad Mahdi Mohammadi (42719)

Date: (Latest Draft): 2026-07-18

Document Type: (Notes)

Table of Contents

1	Pseudo Random Binary Sequence	3
1.1	Structural Implementation of PRBS	3
1.2	Behavioral Implementation of PRBS	3
2	2-bit up/down counter	4
2.1	Structural Implementation of 2-bit up counter	4
2.2	Behavioral Implementation of 2-bit up/down counter	5
3	2-bit up/down counter	6
3.1	Structural Implementation of 2-bit up/down counter	6
3.2	Behavioral Implementation of 2-bit up/down counter	6
4	2-bit counter	7
4.1	Structural Implementation of 2-bit counter	7
4.2	Behavioral Implementation of 2-bit counter	8
5	Bidirectional Shift Register	8
5.1	Structural Implementation of Bidirectional Shift Register	8
5.2	Behavioral Implementation of Bidirectional Shift Register	9
6	Sequence Detector	10
6.1	Structural Implementation of Sequence Detector	10
6.2	Behavioral Implementation of Sequence Detector	11
7	4-bit Shift Register	11
7.1	Structural Implementation of 4-bit Shift Register	11
7.2	Behavioral Implementation of 4-bit Shift Register	12
8	3-bit Complementer	13
8.1	Structural Implementation of 3-bit Complementer	13
8.2	Behavioral Implementation of 3-bit Complementer	13
9	5-bit Variable Shift Register	14
9.1	Structural Implementation of 5-bit Variable Shift Register	14
9.2	Behavioral Implementation of 5-bit Variable Shift Register	15
10	Pattern Detector	16
10.1	Structural Implementation of Pattern Detector	16
10.2	Behavioral Implementation of Pattern Detector	16
11	UART Transmitter	18
12	Inverter Pulse Generator	19
13	PWM Generator	21
14	Variable Frequency Generator	22
15	Minimum and Maximum Finder of stream of numbers	23
16	CRC Generator	25
17	UART Transmission Protocol	26
18	Chirp Signal Generator	28
19	Sign Change Counter	29

1 Psuedo Random Binary Sequence

1.1 Structural Implementation of PRBS

this circuit is the PRBS (pseudo random binary sequence generator 14_2), if u = '0', then it will generate the prbs sequence 1, 8, 4, 2, 9, 12, 6, 11, 5, 10, 13, 14, 15, 7, 3.

if u = '1', then it will shift the number '1' inside the shift register and this vhdl code shows the structural implementation.

```
1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3  USE ieee.numeric_std.ALL;
4
5  ENTITY s14_st1 IS
6      PORT (
7          clk : IN STD_LOGIC;
8          u : IN STD_LOGIC;
9          x : OUT STD_LOGIC
10     );
11 END ENTITY;
12
13 ARCHITECTURE behavioral OF s14_st1 IS
14     SIGNAL q : STD_LOGIC_VECTOR(3 DOWNTO 0) := "0001";
15     SIGNAL a : STD_LOGIC;
16 BEGIN
17     -- flipflops
18     ff : PROCESS (clk)
19     BEGIN
20         IF rising_edge(clk) THEN
21             q(0) <= q(1);
22             q(1) <= q(2);
23             q(2) <= q(3);
24             q(3) <= a;
25         END IF;
26     END PROCESS;
27
28     a <= u OR (q(1) XOR q(0));
29     x <= q(0);
30
31 END ARCHITECTURE behavioral;
```

1.2 Behavioral Implementation of PRBS

And the following code shows the behavioral implementation of the PRBS.

```
1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3  USE ieee.numeric_std.ALL;
4
5  ENTITY s14_st1_bhv IS
6      PORT (
7          clk : IN STD_LOGIC;
8          u : IN STD_LOGIC;
9          x : OUT STD_LOGIC
10     );
11 END ENTITY;
12
13 ARCHITECTURE behavioral OF s14_st1_bhv IS
14     SIGNAL q : STD_LOGIC_VECTOR(3 DOWNTO 0) := "0001";
15
16 BEGIN
17     -- sequential process
18     seq_p : PROCESS (clk)
19
```

```

20     VARIABLE a : STD_LOGIC;
21 BEGIN
22     IF rising_edge(clk) THEN
23         IF u = '1' THEN
24             q <= '1' & q(3 DOWNTO 1);
25         ELSE
26             IF q(1) = q(0) THEN
27                 q <= '0' & q(3 DOWNTO 1);
28             ELSE
29                 q <= '1' & q(3 DOWNTO 1);
30             END IF;
31         END IF;
32     END IF;
33 END PROCESS;
34
35 x <= q(0);
36 END ARCHITECTURE behavioral;

```

2 2-bit up/down counter

2.1 Structural Implementation of 2-bit up counter

this circuit is a 2-bit up counter that counts up and down. it counts the rising edge of the input signal s. The following code shows the structural implementation of the 2-bit up counter.

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3  USE ieee.numeric_std.ALL;
4
5  ENTITY s14_st2 IS
6      PORT (
7          clk : IN STD_LOGIC;
8          s : IN STD_LOGIC;
9          x : OUT STD_LOGIC_VECTOR(1 DOWNTO 0)
10     );
11 END ENTITY;
12
13 ARCHITECTURE behavioral OF s14_st2 IS
14     SIGNAL q : STD_LOGIC_VECTOR(3 DOWNTO 0) := x"0";
15     SIGNAL a, b, c, d, e, f : STD_LOGIC;
16
17 BEGIN
18     -- combinational logic
19     a <= q(3) AND (NOT q(2));
20     b <= a XOR q(0);
21     c <= (NOT a) AND q(1);
22     d <= q(1) AND (NOT q(0));
23     e <= a AND (NOT q(1)) AND q(0);
24     f <= c OR d OR e;
25
26     -- outputs
27     x(0) <= q(0);
28     x(1) <= q(1);
29
30     -- flipflops
31     ff : PROCESS (clk)
32     BEGIN
33         IF rising_edge(clk) THEN
34             q(0) <= b;
35             q(1) <= f;
36             q(2) <= q(3);
37             q(3) <= s;
38         END IF;

```

```

39     END PROCESS;
40 END ARCHITECTURE behavioral;

```

2.2 Behavioral Implementation of 2-bit up/down counter

And the following code shows the behavioral implementation of the 2-bit up counter of the rising edge of signal s.

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3  USE ieee.numeric_std.ALL;
4
5  ENTITY s14_st2_bhv IS
6      PORT (
7          clk : IN STD_LOGIC;
8          s   : IN STD_LOGIC;
9          x   : OUT STD_LOGIC_VECTOR(1 DOWNTO 0)
10     );
11 END ENTITY;
12
13 ARCHITECTURE behavioral OF s14_st2_bhv IS
14     SIGNAL q : STD_LOGIC_VECTOR(1 DOWNTO 0) := "00";
15     SIGNAL s_prev : STD_LOGIC := '0';
16     SIGNAL count : unsigned(1 DOWNTO 0) := "00";
17
18 BEGIN
19
20     -- flipflops
21     ff : PROCESS (clk, s)
22     BEGIN
23         IF rising_edge(clk) THEN
24             IF (s = '1') AND (s_prev = '0') THEN
25                 count <= count + 1;
26             END IF;
27             s_prev <= s;
28         END IF;
29     END PROCESS;
30
31     x <= STD_LOGIC_VECTOR(count);
32 END ARCHITECTURE behavioral;

```

3 2-bit up/down counter

3.1 Structural Implementation of 2-bit up/down counter

this circuit is a 2-bit up-down counter that counts up and down. The output x represents the current count value of the counter. and this circuit is implemented in vhdl in structural style.

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3  USE ieee.numeric_std.ALL;
4
5  ENTITY s15_2 IS
6      PORT (
7          clk : IN STD_LOGIC;
8          x : OUT STD_LOGIC_VECTOR(1 DOWNTO 0)
9      );
10 END ENTITY;
11
12 ARCHITECTURE behavioral OF s15_2 IS
13     SIGNAL q : STD_LOGIC_VECTOR(2 DOWNTO 0) := "000";
14     SIGNAL a, b, c, d : STD_LOGIC;
15
16 BEGIN
17
18     -- flipflops
19     ff : PROCESS (clk)
20     BEGIN
21         IF rising_edge(clk) THEN
22             q(0) <= NOT q(0);
23             q(1) <= d;
24             -- q(1) <= ((NOT q(2) AND q(1))) OR ((NOT q(2)) AND q(0)) OR (q(1) AND
25             q(0));
26             q(2) <= q(1);
27         END IF;
28     END PROCESS;
29
30     -- combinational logic
31     a <= ((NOT q(2) AND q(1)));
32     b <= ((NOT q(2)) AND q(0));
33     c <= (q(1) AND q(0));
34     d <= a OR b OR c;
35
36     -- output logic
37     x <= q(1 DOWNTO 0);
38 END ARCHITECTURE behavioral;

```

3.2 Behavioral Implementation of 2-bit up/down counter

And the following code shows the behavioral implementation of the 2-bit up-down counter.

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3  USE ieee.numeric_std.ALL;
4
5  ENTITY s15_2_bhv IS
6      PORT (
7          clk : IN STD_LOGIC;
8          x : OUT STD_LOGIC_VECTOR(1 DOWNTO 0)
9      );
10 END ENTITY;
11
12 ARCHITECTURE behavioral OF s15_2_bhv IS
13     SIGNAL count : unsigned(1 DOWNTO 0) := "00";
14     SIGNAL cnt_up : STD_LOGIC := '1';

```

```

15
16 BEGIN
17
18   -- flipflops
19   up_down_count : PROCESS (clk)
20   BEGIN
21     IF rising_edge(clk) THEN
22       IF cnt_up = '1' THEN
23         IF count = "11" THEN
24           count <= count + 1;
25           cnt_up <= '0';
26         ELSE
27           count <= count + 1;
28         END IF;
29       ELSE
30         IF count = "00" THEN
31           count <= count - 1;
32           cnt_up <= '1';
33         ELSE
34           count <= count - 1;
35         END IF;
36       END IF;
37     END IF;
38   END PROCESS up_down_count;
39   -- output logic
40   x <= STD_LOGIC_VECTOR(count);
41
42 END ARCHITECTURE behavioral;

```

4 2-bit counter

4.1 Structural Implementation of 2-bit counter

this circuit is a 2-bit counter that counts up when u = '1' and pause the counting when u = '0'. The output x represents the current count value of the counter. and this circuit is implemented in vhd1 in structural style.

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3  USE ieee.numeric_std.ALL;
4
5  ENTITY s16_1 IS
6    PORT (
7      clk : IN STD_LOGIC;
8      u   : IN STD_LOGIC;
9      x   : OUT STD_LOGIC_VECTOR(1 DOWNTO 0)
10    );
11 END ENTITY;
12
13 ARCHITECTURE behavioral OF s16_1 IS
14   SIGNAL q : STD_LOGIC_VECTOR(1 DOWNTO 0) := "00";
15   SIGNAL a, b, c, d, e, f, g : STD_LOGIC;
16
17 BEGIN
18
19   -- combinational logic
20   a <= u OR q(1);
21   b <= q(1) OR q(0);
22   c <= (NOT u) OR (NOT q(0)) OR (NOT q(1));
23   d <= a AND b AND c;
24
25   e <= (NOT u) AND q(0);
26   f <= (NOT q(0)) AND u;

```

```

27     g <= e OR f;
28
29     -- output logic
30     x <= q(1 DOWNTO 0);
31
32     -- flipflops
33     ff : PROCESS (clk)
34     BEGIN
35         IF rising_edge(clk) THEN
36             q(0) <= g;
37             q(1) <= d;
38         END IF;
39     END PROCESS;
40
41 END ARCHITECTURE behavioral;

```

4.2 Behavioral Implementation of 2-bit counter

And the following code shows the behavioral implementation of the 2-bit counter.

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3  USE ieee.numeric_std.ALL;
4
5  ENTITY s16_1_bhv IS
6      PORT (
7          clk : IN STD_LOGIC;
8          u   : IN STD_LOGIC;
9          x   : OUT STD_LOGIC_VECTOR(1 DOWNTO 0)
10     );
11 END ENTITY;
12
13 ARCHITECTURE behavioral OF s16_1_bhv IS
14     SIGNAL q : STD_LOGIC_VECTOR(1 DOWNTO 0) := "00";
15     SIGNAL cnt : unsigned(1 DOWNTO 0) := "00";
16
17 BEGIN
18
19     -- flipflops
20     ff : PROCESS (clk)
21     BEGIN
22         IF rising_edge(clk) THEN
23             IF u = '0' THEN
24                 cnt <= cnt;
25             ELSE
26                 cnt <= cnt + 1;
27             END IF;
28         END IF;
29     END PROCESS;
30
31     -- output
32     x <= STD_LOGIC_VECTOR(cnt);
33
34 END ARCHITECTURE behavioral;

```

5 Bidirectional Shift Register

5.1 Structural Implementation of Bidirectional Shift Register

this circuit is a bidirectional shift register with 4 bits. The input s controls the direction of the shift. When s = '0', the register shifts to the right, and when s = '1', it shifts to the left. The outputs x and y represent the second and first bits of the register, respectively.

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3  USE ieee.numeric_std.ALL;
4
5  ENTITY s16_3a IS
6      PORT (
7          clk : IN STD_LOGIC;
8          s : IN STD_LOGIC;
9          x, y : OUT STD_LOGIC
10     );
11 END ENTITY;
12
13 ARCHITECTURE behavioral OF s16_3a IS
14     SIGNAL q : STD_LOGIC_VECTOR(3 DOWNTO 0) := "0010";
15     SIGNAL a, b, c, d : STD_LOGIC := '0';
16
17     SIGNAL y_in : STD_LOGIC_VECTOR(1 DOWNTO 0) := (OTHERS => '0');
18 BEGIN
19
20     -- flipflops
21     ff : PROCESS (clk)
22     BEGIN
23         IF rising_edge(clk) THEN
24             q(3) <= a;
25             q(2) <= b;
26             q(1) <= c;
27             q(0) <= d;
28         END IF;
29     END PROCESS;
30
31     -- combinational logic
32     a <= ((NOT s) AND q(0)) OR (s AND q(2));
33     b <= ((NOT s) AND q(3)) OR (s AND q(1));
34     c <= ((NOT s) AND q(2)) OR (s AND q(0));
35     d <= ((NOT s) AND q(1)) OR (s AND q(3));
36
37     -- outputs
38     x <= q(1);
39     y <= q(0);
40
41     y_in <= q(1) & q(0);
42
43 END ARCHITECTURE behavioral;

```

5.2 Behavioral Implementation of Bidirectional Shift Register

And the following code shows the behavioral implementation of the bidirectional shift register.

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3  USE ieee.numeric_std.ALL;
4
5  ENTITY s16_2_bhv IS
6      PORT (
7          clk : IN STD_LOGIC;
8          s : IN STD_LOGIC;
9          x : OUT STD_LOGIC;
10         y : OUT STD_LOGIC
11     );
12 END ENTITY;
13
14 ARCHITECTURE behavioral OF s16_2_bhv IS
15     SIGNAL q : STD_LOGIC_VECTOR(3 DOWNTO 0) := x"1";
16 BEGIN
17
18     -- sequential process

```

```

19  seq_p : PROCESS (clk)
20  BEGIN
21      IF rising_edge(clk) THEN
22          IF s = '0' THEN
23              q <= q(0) & q(3 DOWNT0 1);
24          ELSE
25              q <= q(2 DOWNT0 0) & q(3);
26          END IF;
27      END IF;
28  END PROCESS;
29
30  -- output
31  x <= q(1);
32  y <= q(0);
33
34  END ARCHITECTURE behavioral;

```

6 Sequence Detector

6.1 Structural Implementation of Sequence Detector

this circuit is a 5-bit shift register with a feedback loop that generates a specific output based on the input signal 'u' and the current state of the register. The output 'y' is determined by the logic conditions defined in the architecture. the output becomes when the register sequence reaches "00101"

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3  USE ieee.numeric_std.ALL;
4
5  ENTITY s21_3a IS
6      PORT (
7          clk : IN STD_LOGIC;
8          u : IN STD_LOGIC;
9          y : OUT STD_LOGIC
10     );
11  END ENTITY;
12
13  ARCHITECTURE behavioral OF s21_3a IS
14      SIGNAL q : STD_LOGIC_VECTOR(4 DOWNT0 0) := "00000";
15      SIGNAL a, b, c, d, e, f, g : STD_LOGIC := '0';
16  BEGIN
17
18      -- combinational circuits
19      a <= u AND g;
20      b <= q(4) AND g;
21      c <= q(3) AND g;
22      d <= q(2) AND g;
23      e <= q(1) AND g;
24      f <= q(4) AND (NOT q(3)) AND q(2) AND (NOT q(1)) AND (NOT q(0));
25      g <= NOT f;
26
27      -- output logic
28      y <= f;
29
30      -- flipflops
31      ff : PROCESS (clk)
32      BEGIN
33          IF rising_edge(clk) THEN
34              q(4) <= a;
35              q(3) <= b;
36              q(2) <= c;
37              q(1) <= d;
38              q(0) <= e;

```

```

39     END IF;
40     END PROCESS;
41 END ARCHITECTURE behavioral;

```

6.2 Behavioral Implementation of Sequence Detector

And the following code shows the behavioral implementation of the sequence detector.

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3  USE ieee.numeric_std.ALL;
4
5  ENTITY s21_b IS
6      PORT (
7          clk : IN STD_LOGIC;
8          u : IN STD_LOGIC;
9          y : OUT STD_LOGIC
10     );
11 END ENTITY;
12
13 ARCHITECTURE behavioral OF s21_b IS
14     SIGNAL q : STD_LOGIC_VECTOR(4 DOWNTO 0) := "00000";
15 BEGIN
16
17     -- sequential process
18     seq_p : PROCESS (clk)
19     BEGIN
20         IF rising_edge(clk) THEN
21             -- making the shift register
22             q <= u & q(4 DOWNTO 1);
23             IF q = "10100" THEN
24                 y <= '1';
25             ELSE
26                 y <= '0';
27             END IF;
28         END IF;
29     END PROCESS;
30
31 END ARCHITECTURE behavioral;

```

7 4-bit Shift Register

7.1 Structural Implementation of 4-bit Shift Register

this circuit is a 4-bit shift register that will become "0000", when the data inside the register becomes "1110", then the output will become "0000".

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3  USE ieee.numeric_std.ALL;
4
5  ENTITY s22_3a IS
6      PORT (
7          clk : IN STD_LOGIC;
8          y : OUT STD_LOGIC_VECTOR(3 DOWNTO 0)
9      );
10 END ENTITY;
11
12 ARCHITECTURE behavioral OF s22_3a IS
13     SIGNAL q : STD_LOGIC_VECTOR(3 DOWNTO 0) := x"6";
14     SIGNAL a, b, c, d, e : STD_LOGIC;
15 BEGIN
16

```

```

17  --flipflops
18  ff : PROCESS (clk)
19  BEGIN
20      IF rising_edge(clk) THEN
21          q(0) <= a;
22          q(1) <= b;
23          q(2) <= c;
24          q(3) <= d;
25      END IF;
26  END PROCESS ff;
27
28  --combinational process
29  a <= (NOT q(3)) AND e;
30  b <= q(0) AND e;
31  c <= q(1) AND e;
32  d <= q(2) AND e;
33  e <= NOT (q(3) AND q(3)) AND q(2) AND q(1) AND (NOT q(0));
34
35  --the output
36  y <= q;
37
38  END ARCHITECTURE behavioral;

```

7.2 Behavioral Implementation of 4-bit Shift Register

And the following code shows the behavioral implementation of the 4-bit shift register.

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3  USE ieee.numeric_std.ALL;
4
5  ENTITY s22_b IS
6      PORT (
7          clk : IN STD_LOGIC;
8          y : OUT STD_LOGIC_VECTOR(3 DOWNTO 0)
9      );
10  END ENTITY;
11
12  ARCHITECTURE behavioral OF s22_b IS
13      SIGNAL q : STD_LOGIC_VECTOR(3 DOWNTO 0) := "1101";
14      SIGNAL yin : STD_LOGIC_VECTOR(3 DOWNTO 0) := (OTHERS => '0');
15  BEGIN
16
17      -- sequential process
18      seq_p : PROCESS (clk)
19      BEGIN
20          IF rising_edge(clk) THEN
21              -- the data shift inside the register
22              q <= q(0) & q(3 DOWNTO 1);
23              IF q = "1110" THEN
24                  yin <= (OTHERS => '0');
25              ELSE
26                  yin <= q;
27              END IF;
28          END IF;
29      END PROCESS;
30
31      y <= yin;
32
33  END ARCHITECTURE behavioral;

```

8 3-bit Complementer

8.1 Structural Implementation of 3-bit Complementer

this circuit is used to calculate the complement of a 3-bit number. The circuit has a control input u , which determines whether the output y is equal to the input x (when $u=1$) or the complement of the input x (when $u=0$). The circuit uses flip-flops to store the state of the output y , and combinational logic to calculate the next state based on the current state and the input x .

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3  USE ieee.numeric_std.ALL;
4
5  ENTITY s23_3a IS
6      PORT (
7          clk : IN STD_LOGIC;
8          u : IN STD_LOGIC;
9          x : IN STD_LOGIC_VECTOR(2 DOWNTO 0);
10         y : OUT STD_LOGIC_VECTOR(2 DOWNTO 0)
11     );
12 END ENTITY;
13
14 ARCHITECTURE behavioral OF s23_3a IS
15     SIGNAL q : STD_LOGIC_VECTOR(2 DOWNTO 0) := (OTHERS => '0');
16     SIGNAL a, b, c : STD_LOGIC;
17
18 BEGIN
19
20     --flipflops
21     ff : PROCESS (clk)
22     BEGIN
23         IF rising_edge(clk) THEN
24             q(2) <= a;
25             q(1) <= b;
26             q(0) <= c;
27         END IF;
28     END PROCESS ff;
29
30     --combinational logic
31     a <= (u AND x(2)) OR ((NOT u) AND ((q(2) AND (NOT q(1)) AND (NOT q(0))) OR ((NOT
32     q(2) AND q(0))) OR ((NOT q(2)) AND q(1))));
33     b <= (u AND x(1)) OR ((NOT u) AND (q(1) XOR q(0)));
34     c <= (u AND x(0)) OR ((NOT u) AND q(0));
35
36     --output combination logic
37     y <= q;
38
39 END ARCHITECTURE behavioral;

```

8.2 Behavioral Implementation of 3-bit Complementer

And the following code shows the behavioral implementation of the 3-bit complementer.

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3  USE ieee.numeric_std.ALL;
4
5  ENTITY s23_3b IS
6      PORT (
7          clk : IN STD_LOGIC;
8          u : IN STD_LOGIC;
9          x : IN STD_LOGIC_VECTOR(2 DOWNTO 0);
10         y : OUT STD_LOGIC_VECTOR(2 DOWNTO 0)
11     );

```

```

12 END ENTITY;
13
14 ARCHITECTURE behavioral OF s23_3b IS
15     SIGNAL q : STD_LOGIC_VECTOR(2 DOWNTO 0) := (OTHERS => '0');
16     SIGNAL x_in : INTEGER := 0;
17 BEGIN
18     x_in <= to_integer(signed(x));
19
20     -- sequential process
21     seq_p : PROCESS (clk)
22     BEGIN
23         IF rising_edge(clk) THEN
24             IF u = '1' THEN
25                 q <= STD_LOGIC_VECTOR(to_signed(x_in, q'length));
26             ELSE
27                 -- complement logic
28                 q <= STD_LOGIC_VECTOR(to_signed(-x_in, q'length));
29             END IF;
30         END IF;
31     END PROCESS;
32
33     -- combinational process
34     y <= q;
35
36 END ARCHITECTURE behavioral;
37

```

9 5-bit Variable Shift Register

9.1 Structural Implementation of 5-bit Variable Shift Register

the circuit is a 5-bit variable shift register. when sel = 0, the register shifts right and the input d_in is shifted into the least significant bit. when sel = 1, the register shift right the d_in twice into the least significant bit. the output is the current state of the register. this is the structural implementation of the circuit.

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3  USE ieee.numeric_std.ALL;
4
5  ENTITY s24 IS
6      PORT (
7          clk : IN STD_LOGIC;
8          d_in : IN STD_LOGIC;
9          sel : IN STD_LOGIC;
10         y : OUT STD_LOGIC_VECTOR(4 DOWNTO 0)
11     );
12 END ENTITY;
13
14 ARCHITECTURE behavioral OF s24 IS
15     SIGNAL q : STD_LOGIC_VECTOR(4 DOWNTO 0) := "00000";
16     SIGNAL a, b, c, d, e, f, g, h, i, j, k, l : STD_LOGIC;
17 BEGIN
18     -- combinational circuits
19     a <= a AND (NOT sel);
20     b <= d_in AND sel;
21     c <= a OR b;
22
23     d <= q(1) AND (NOT sel);
24     e <= q(0) AND sel;
25     f <= d OR e;
26
27     g <= q(2) AND (NOT sel);
28

```

```

29  h <= q(1) AND sel;
30  i <= g OR h;
31
32  j <= q(3) AND (NOT sel);
33  k <= q(2) AND sel;
34  l <= j OR k;
35
36  -- outputs
37  y <= q;
38
39  -- flipflops
40  ff : PROCESS (clk)
41  BEGIN
42      IF rising_edge(clk) THEN
43          q(0) <= d_in;
44          q(1) <= c;
45          q(2) <= f;
46          q(3) <= i;
47          q(4) <= l;
48      END IF;
49  END PROCESS ff;
50
51  END behavioral;

```

9.2 Behavioral Implementation of 5-bit Variable Shift Register

And the following code shows the behavioral implementation of the 5-bit variable shift register.

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3  USE ieee.numeric_std.ALL;
4
5  ENTITY s24_3b IS
6      PORT (
7          clk : IN STD_LOGIC;
8          d_in : IN STD_LOGIC;
9          sel : IN STD_LOGIC;
10         y : OUT STD_LOGIC_VECTOR(4 DOWNTO 0)
11     );
12  END ENTITY;
13
14  ARCHITECTURE behavioral OF s24_3b IS
15      SIGNAL q : STD_LOGIC_VECTOR(4 DOWNTO 0);
16  BEGIN
17
18      -- s_reg process
19      s_reg_p : PROCESS (clk)
20      BEGIN
21          IF rising_edge(clk) THEN
22              IF sel = '1' THEN
23                  q <= q(3 DOWNTO 0) & d_in;
24              ELSE
25                  IF d_in = '0' THEN
26                      q <= q(2 DOWNTO 0) & "00";
27                  ELSE
28                      q <= q(2 DOWNTO 0) & "11";
29                  END IF;
30              END IF;
31          END IF;
32      END PROCESS;
33
34  END ARCHITECTURE behavioral;

```

10 Pattern Detector

10.1 Structural Implementation of Pattern Detector

this circuit is a pattern detector, and detects how many times the pattern “010” appears in the input stream.

```

1  LIBRARY IEEE;
2  USE IEEE.STD_LOGIC_1164.ALL;
3  USE IEEE.NUMERIC_STD.ALL;
4
5  ENTITY s25_3st IS
6      PORT (
7          clk : IN STD_LOGIC;
8          d_in : IN STD_LOGIC;
9          y : OUT STD_LOGIC_VECTOR(1 DOWNTO 0)
10     );
11 END ENTITY;
12
13 ARCHITECTURE structural OF s25_3st IS
14     SIGNAL q : STD_LOGIC_VECTOR(4 DOWNTO 0) := "00000";
15     SIGNAL a, b, c, d, e, f, g, h : STD_LOGIC;
16 BEGIN
17
18     -- sequential circuits
19     ff : PROCESS (clk)
20     BEGIN
21         IF rising_edge(clk) THEN
22             q(4) <= d_in;
23             q(3) <= q(4);
24             q(2) <= q(3);
25             q(1) <= e;
26             q(0) <= h;
27         END IF;
28     END PROCESS ff;
29
30     -- combinational circuits
31     a <= (NOT q(4)) AND q(3) AND (NOT q(2));
32     b <= (NOT a) AND q(1);
33     c <= q(1) AND (NOT q(0));
34     d <= a AND (NOT q(1)) AND q(0);
35     e <= b OR c OR d;
36
37     f <= (NOT a) AND q(0);
38     g <= a AND (NOT q(0));
39     h <= f OR g;
40
41     -- outputs
42     y <= q(1 DOWNTO 0);
43 END ARCHITECTURE structural;

```

10.2 Behavioral Implementation of Pattern Detector

And the following code shows the behavioral implementation of the pattern detector.

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3  USE ieee.numeric_std.ALL;
4
5  ENTITY s253b IS
6      PORT (
7          clk : IN STD_LOGIC;
8          d_in : IN STD_LOGIC;
9          y : OUT STD_LOGIC_VECTOR(1 DOWNTO 0)

```

```

10     );
11 END ENTITY;
12
13 ARCHITECTURE behavioral OF s253b IS
14
15     SIGNAL count : unsigned(1 DOWNT0 0) := "00";
16     SIGNAL readbuf : STD_LOGIC_VECTOR(2 DOWNT0 0) := "000";
17
18 BEGIN
19
20     seq_det_p : PROCESS (clk)
21     BEGIN
22         IF rising_edge(clk) THEN
23
24             -- Check the sequence including the current input bit
25             IF readbuf = "010" THEN
26                 count <= count + 1;
27             END IF;
28
29             -- Shift the current input into the buffer
30             readbuf <= readbuf(1 DOWNT0 0) & d_in;
31
32         END IF;
33     END PROCESS seq_det_p;
34
35     y <= STD_LOGIC_VECTOR(count);
36
37 END ARCHITECTURE behavioral;

```

11 UART Transmitter

The following code shows the implementation of a UART transmitter in VHDL. (16_1)

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3  USE ieee.numeric_std.ALL;
4
5  ENTITY s16_1 IS
6      PORT (
7          clk : IN STD_LOGIC;
8          reset : IN STD_LOGIC;
9          start : IN STD_LOGIC;
10         txdata : IN STD_LOGIC_VECTOR(6 DOWNTO 0);
11         TxD : OUT STD_LOGIC;
12         done : OUT STD_LOGIC
13     );
14 END ENTITY s16_1;
15
16 ARCHITECTURE behavioral OF s16_1 IS
17     TYPE state_type IS (idle, state0, state_trans, state_parity, state_stopbit);
18     SIGNAL state, next_state : state_type;
19     SUBTYPE counter_type IS INTEGER RANGE 0 TO 6;
20     SIGNAL cnt, cnt_i : counter_type;
21
22 BEGIN
23
24     -- sequential process
25     seq_p : PROCESS (clk)
26     BEGIN
27         IF rising_edge(clk) THEN
28             IF reset = '1' THEN
29                 state <= idle;
30                 cnt <= 0;
31             ELSE
32                 state <= next_state;
33                 cnt <= cnt_i;
34             END IF;
35         END IF;
36     END PROCESS;
37
38     -- combinational process
39     comb_p : PROCESS (state, start, cnt)
40     BEGIN
41         -- default cases
42         done <= '0';
43         next_state <= state;
44         TxD <= '0';
45
46         CASE state IS
47             WHEN idle =>
48                 TxD <= '1';
49                 done <= '1';
50                 IF start = '1' THEN
51                     next_state <= state0;
52                 END IF;
53             WHEN state0 =>
54                 cnt_i <= 6;
55                 next_state <= state_trans;
56             WHEN state_trans =>
57                 TxD <= txdata(cnt);
58                 IF cnt = 0 THEN
59                     next_state <= state_parity;
60                 ELSE
61                     cnt_i <= cnt - 1;
62                 END IF;

```

```

63         WHEN state_parity =>
64             TxData <= txdata(0) XOR txdata(1) XOR txdata(1) XOR txdata(2) XOR
txdata(3) XOR txdata(4) XOR txdata(5) XOR txdata(6);
65             next_state <= state_stopbit;
66         WHEN state_stopbit =>
67             TxData <= '1';
68             next_state <= idle;
69     END CASE;
70 END PROCESS;
71
72 END ARCHITECTURE behavioral;

```

12 Inverter Pulse Generator

The following code shows the implementation of an inverter pulse generator in VHDL. (14_1)

```

1  -----
2  -- Engineer: Mohammad Mahdi Mohammadi
3  --
4  -- Create Date: 07/16/2026 11:28:10 AM
5  -- Design Name:
6  -- Module Name: s14 - Behavioral
7  -- Description:
8  --
9  -- Dependencies:
10 --
11 -- Revision:
12 -- Revision 0.01 - File Created
13 -- Additional Comments:
14 --
15 -----
16 LIBRARY IEEE;
17 USE IEEE.STD_LOGIC_1164.ALL;
18 USE IEEE.NUMERIC_STD.ALL;
19 ENTITY s14 IS
20     PORT (
21         clk : IN STD_LOGIC;
22         reset : IN STD_LOGIC;
23         ss : IN STD_LOGIC;
24         s1, s2, s3 : OUT STD_LOGIC
25     );
26 END s14;
27
28 ARCHITECTURE Behavioral OF s14 IS
29     TYPE state_type IS (idle, st0, st1, st2, st3, st4, st5, off0, off1, off2, off3,
off4, off5);
30     SIGNAL state, next_state : state_type;
31
32 BEGIN
33
34     -- sequential process
35     -- for calculation of the next state.
36     seq_p : PROCESS (clk, reset)
37     BEGIN
38         IF rising_edge(clk) THEN
39             IF reset = '1' THEN
40                 state <= idle;
41             ELSE
42                 state <= next_state;
43             END IF;
44         END IF;
45     END PROCESS seq_p;
46
47     -- combinational circuit

```

```

48  comb_p : PROCESS (state, ss)
49  BEGIN
50      -- default cases
51      next_state <= state;
52
53      CASE state IS
54          WHEN idle =>
55              IF ss = '1' THEN
56                  next_state <= st0;
57              END IF;
58          WHEN st0 =>
59              s1 <= '1';
60              s2 <= '0';
61              s3 <= '0';
62              IF ss = '1' THEN
63                  next_state <= st1;
64              ELSE
65                  next_state <= off0;
66              END IF;
67          WHEN st1 =>
68              s1 <= '1';
69              s2 <= '1';
70              s3 <= '0';
71              IF ss = '1' THEN
72                  next_state <= st2;
73              ELSE
74                  next_state <= off1;
75              END IF;
76          WHEN st2 =>
77              s1 <= '0';
78              s2 <= '1';
79              s3 <= '0';
80              IF ss = '1' THEN
81                  next_state <= st3;
82              ELSE
83                  next_state <= off2;
84              END IF;
85          WHEN st3 =>
86              s1 <= '0';
87              s2 <= '1';
88              s3 <= '1';
89              IF ss = '1' THEN
90                  next_state <= st4;
91              ELSE
92                  next_state <= off3;
93              END IF;
94          WHEN st4 =>
95              s1 <= '0';
96              s2 <= '0';
97              s3 <= '1';
98              IF ss = '1' THEN
99                  next_state <= st5;
100             ELSE
101                 next_state <= off4;
102             END IF;
103          WHEN st5 =>
104              s1 <= '1';
105              s2 <= '0';
106              s3 <= '1';
107              IF ss = '1' THEN
108                  next_state <= st0;
109              ELSE
110                  next_state <= off5;
111              END IF;

```

```

112         WHEN off0 =>
113             IF ss = '1' THEN
114                 next_state <= st1;
115             END IF;
116         WHEN off1 =>
117             IF ss = '1' THEN
118                 next_state <= st2;
119             END IF;
120         WHEN off2 =>
121             IF ss = '1' THEN
122                 next_state <= st3;
123             END IF;
124         WHEN off3 =>
125             IF ss = '1' THEN
126                 next_state <= st4;
127             END IF;
128         WHEN off4 =>
129             IF ss = '1' THEN
130                 next_state <= st5;
131             END IF;
132         WHEN off5 =>
133             IF ss = '1' THEN
134                 next_state <= st0;
135             END IF;
136     END CASE;
137 END PROCESS comb_p;
138
139 END Behavioral;

```

13 PWM Generator

The following code shows the implementation of a PWM generator in VHDL.

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3  USE ieee.numeric_std.ALL;
4
5  ENTITY s15_2 IS
6      GENERIC (
7          NBITS : INTEGER := 12
8      );
9      PORT (
10         clk : IN STD_LOGIC;
11         reset : IN STD_LOGIC;
12         enable : IN STD_LOGIC;
13         pval : IN STD_LOGIC_VECTOR(NBITS - 1 DOWNTO 0);
14         pwmout : OUT STD_LOGIC
15     );
16 END ENTITY;
17
18 ARCHITECTURE behavioral OF s15_2 IS
19     SIGNAL counter : unsigned(NBITS - 1 DOWNTO 0) := (OTHERS => '0');
20
21 BEGIN
22
23     -- free running counter
24     cnt_p : PROCESS (clk, reset)
25     BEGIN
26         IF rising_edge(clk) THEN
27             IF reset = '1' THEN
28                 counter <= (OTHERS => '0');
29             ELSE
30                 counter <= counter + 1;
31             END IF;

```

```

32     END IF;
33 END PROCESS cnt_p;
34
35 -- PWM process
36 pwm_p : PROCESS (clk, enable, pval, counter, reset)
37 BEGIN
38     IF rising_edge(clk) THEN
39         IF reset = '1' THEN
40             pwmout <= '0';
41         ELSIF enable = '0' THEN
42             pwmout <= '0';
43         ELSIF counter > unsigned(pval) THEN
44             pwmout <= '0';
45         ELSE
46             pwmout <= '1';
47         END IF;
48     END IF;
49 END PROCESS pwm_p;
50
51 END ARCHITECTURE behavioral;

```

14 Variable Frequency Generator

The following code shows the implementation of a variable frequency generator in VHDL.

```

1  ENTITY s16 IS
2      PORT (
3          clk : IN STD_LOGIC;
4          reset : IN STD_LOGIC;
5          start : IN STD_LOGIC;
6          incr : IN STD_LOGIC;
7          stop : IN STD_LOGIC;
8          clk_out : OUT STD_LOGIC
9      );
10 END ENTITY;
11
12 ARCHITECTURE behavioral OF s16 IS
13     TYPE state_type IS (idle, state_1_L, state_1_H, state_10_L, state_10_H);
14     SIGNAL state, next_state : state_type;
15     SIGNAL timer : INTEGER := 0;
16
17     CONSTANT c1Mhz_coeff : INTEGER := 49;
18     CONSTANT c10Mhz_coeff : INTEGER := 4;
19 BEGIN
20
21     -- sequential process
22     seq_p : PROCESS (clk)
23         VARIABLE cnt : INTEGER := 0;
24     BEGIN
25         IF rising_edge(clk) THEN
26             IF reset = '1' THEN
27                 state <= idle;
28                 cnt := 0;
29             ELSE
30                 IF cnt = timer THEN
31                     state <= next_state;
32                     cnt := 0;
33                 ELSE
34                     cnt := cnt + 1;
35                 END IF;
36             END IF;
37         END IF;
38     END PROCESS;
39

```

```

40  -- combinational process
41  comb_p : PROCESS (state, start, stop, incr)
42  BEGIN
43      -- default cases
44      next_state <= state;
45      timer <= 0;
46      clk_out <= '0';
47
48      CASE state IS
49          WHEN idle =>
50              IF start = '1' THEN
51                  next_state <= state_1_L;
52              END IF;
53          WHEN state_1_L =>
54              timer <= c1Mhz_coeff;
55              IF stop = '1' THEN
56                  next_state <= idle;
57              ELSIF incr = '1' THEN
58                  next_state <= state_10_L;
59              ELSE
60                  next_state <= state_1_H;
61              END IF;
62          WHEN state_1_H =>
63              clk_out <= '1';
64              timer <= c1Mhz_coeff;
65              IF stop = '1' THEN
66                  next_state <= idle;
67              ELSIF incr = '1' THEN
68                  next_state <= state_10_L;
69              ELSE
70                  next_state <= state_1_L;
71              END IF;
72          WHEN state_10_L =>
73              timer <= c10Mhz_coeff;
74              IF stop = '1' THEN
75                  next_state <= idle;
76              ELSE
77                  next_state <= state_10_H;
78              END IF;
79          WHEN state_10_H =>
80              clk_out <= '1';
81              timer <= c10Mhz_coeff;
82              IF stop = '1' THEN
83                  next_state <= idle;
84              ELSE
85                  next_state <= state_10_L;
86              END IF;
87      END CASE;
88  END PROCESS;
89
90  END ARCHITECTURE behavioral;

```

15 Minimum and Maximum Finder of stream of numbers

The following code shows the implementation of a minimum and maximum finder of stream of numbers in VHDL.

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3  USE ieee.numeric_std.ALL;
4
5  ENTITY s21 IS
6      PORT (

```

```

7      clk : IN STD_LOGIC;
8      reset : IN STD_LOGIC;
9      start : IN STD_LOGIC;
10     x_in : IN STD_LOGIC_VECTOR(11 DOWNTO 0);
11     xmin : OUT STD_LOGIC_VECTOR(11 DOWNTO 0);
12     xmax : OUT STD_LOGIC_VECTOR(11 DOWNTO 0);
13     done : OUT STD_LOGIC
14 );
15 END ENTITY s21;
16
17 ARCHITECTURE Behavioral OF s21 IS
18     TYPE state_type IS (idle, running);
19     SIGNAL state : state_type := idle;
20     SUBTYPE int12_t IS INTEGER RANGE -2048 TO 2047;
21     SIGNAL xmin_i, xmax_i : int12_t;
22
23 BEGIN
24
25     -- sequential process
26     seq_p : PROCESS (clk)
27     BEGIN
28         IF rising_edge(clk) THEN
29             IF reset = '1' THEN
30                 state <= idle;
31                 xmin_i <= 0;
32                 xmax_i <= 0;
33             ELSE
34                 -- default cases
35                 done <= '0';
36                 CASE state IS
37                     WHEN idle =>
38                         done <= '1';
39                         IF start = '1' THEN
40                             state <= running;
41                             xmin_i <= to_integer(signed(x_in));
42                             xmax_i <= to_integer(signed(x_in));
43                         END IF;
44                     WHEN running =>
45                         IF to_integer(signed(x_in)) < xmin_i THEN
46                             xmin_i <= to_integer(signed(x_in));
47                         ELSE
48                             xmin_i <= xmin_i;
49                         END IF;
50
51                         IF to_integer(signed(x_in)) > xmax_i THEN
52                             xmax_i <= to_integer(signed(x_in));
53                         ELSE
54                             xmax_i <= xmax_i;
55                         END IF;
56
57                         IF start = '0' THEN
58                             state <= idle;
59                         END IF;
60                     END CASE;
61                 END IF;
62             END IF;
63         END PROCESS;
64
65     -- outputs
66     xmin <= STD_LOGIC_VECTOR(to_signed(xmin_i, xmin'length));
67     xmax <= STD_LOGIC_VECTOR(to_signed(xmax_i, xmax'length));
68
69 END ARCHITECTURE behavioral;

```

16 CRC Generator

The following code shows the implementation of a CRC generator in VHDL.

```

1  -----
2  -- Company:
3  -- Engineer:
4  --
5  -- Create Date: 07/17/2026 02:49:07 PM
6  -- Design Name:
7  -- Module Name: s22 - Behavioral
8  -- Project Name:
9  -- Target Devices:
10 -- Tool Versions:
11 -- Description:
12 --
13 -- Dependencies:
14 --
15 -- Revision:
16 -- Revision 0.01 - File Created
17 -- Additional Comments:
18 --
19 -----
20 LIBRARY IEEE;
21 USE IEEE.STD_LOGIC_1164.ALL;
22 USE IEEE.NUMERIC_STD.ALL;
23 ENTITY s22 IS
24     PORT (
25         clk : IN STD_LOGIC;
26         reset : IN STD_LOGIC;
27         start : IN STD_LOGIC;
28         din : IN STD_LOGIC_VECTOR(7 DOWNTO 0);
29         crc : OUT STD_LOGIC_VECTOR(3 DOWNTO 0);
30         done : OUT STD_LOGIC
31     );
32 END s22;
33
34 ARCHITECTURE Behavioral OF s22 IS
35     TYPE state_type IS (idle, busy);
36     SIGNAL state : state_type;
37     SUBTYPE counter_type IS INTEGER RANGE 0 TO 8;
38     SIGNAL cnt : counter_type := 0;
39
40     -- for the flipflops
41     SIGNAL crc_i : STD_LOGIC_VECTOR(3 DOWNTO 0) := x"0";
42 BEGIN
43
44     -- sequential process
45     seq_p : PROCESS (clk, state, cnt)
46         VARIABLE feedback : STD_LOGIC;
47     BEGIN
48         IF rising_edge(clk) THEN
49             IF reset = '1' THEN
50                 crc_i <= x"0";
51                 cnt <= 0;
52             ELSE
53                 -- default cases
54                 done <= '0';
55
56                 CASE state IS
57                     WHEN idle =>
58                         done <= '1';
59                         IF start = '1' THEN
60                             cnt <= 7;
61                             state <= busy;

```

```

62         crc_i <= x"0";
63     END IF;
64     WHEN busy =>
65         feedback := din(cnt) XOR crc_i(3);
66         crc_i(0) <= feedback;
67         crc_i(1) <= crc_i(0) XOR feedback;
68         crc_i(2) <= crc_i(1);
69         crc_i(3) <= crc_i(2);
70
71         IF cnt = 0 THEN
72             state <= idle;
73         ELSE
74             cnt <= cnt - 1;
75         END IF;
76     END CASE;
77 END IF;
78 END IF;
79 END PROCESS seq_p;
80
81 -- output
82 crc <= crc_i;
83
84 END Behavioral;

```

17 UART Transmission Protocol

The following code shows the implementation of a UART transmission protocol in VHDL.

```

1  -----
2  -- Company:
3  -- Engineer: Mohammad Mahdi Mohammadi
4  --
5  -- Create Date: 07/17/2026 12:01:47 AM
6  -- Design Name:
7  -- Module Name: s23 - Behavioral
8  -- Project Name:
9  -- Target Devices:
10 -- Tool Versions:
11 -- Description: UART serial tranmission protocol
12 --
13 -- Dependencies:
14 --
15 -- Revision:
16 -- Revision 0.01 - File Created
17 -- Additional Comments:
18 --
19 -----
20 LIBRARY IEEE;
21 USE IEEE.STD_LOGIC_1164.ALL;
22 USE IEEE.NUMERIC_STD.ALL;
23
24 ENTITY s23 IS
25     PORT (
26         clk : IN STD_LOGIC;
27         start : IN STD_LOGIC;
28         reset : IN STD_LOGIC;
29         data_in : IN STD_LOGIC_VECTOR(7 DOWNTO 0);
30         y_out : OUT STD_LOGIC
31     );
32 END s23;
33
34 ARCHITECTURE Behavioral OF s23 IS
35     TYPE state_type IS (idle, state_0, state_trans, state_parity, state_stop);
36     SIGNAL state, next_state : state_type;

```

```

37
38     SUBTYPE counter_type IS INTEGER RANGE 0 TO 7;
39     SIGNAL cnt, cnt_i : counter_type;
40     SIGNAL write_buf : STD_LOGIC_VECTOR(7 DOWNTO 0) := (OTHERS => '0');
41     SIGNAL read_buf : STD_LOGIC_VECTOR(7 DOWNTO 0) := (OTHERS => '0');
42
43     SIGNAL y_out_i : STD_LOGIC := '0';
44
45     CONSTANT MAXBITS : INTEGER := 8;
46 BEGIN
47
48     -- sequential process
49     seq_p : PROCESS (clk)
50     BEGIN
51         IF rising_edge(clk) THEN
52             IF reset = '1' THEN
53                 state <= idle;
54                 cnt <= 0;
55             ELSE
56                 IF next_state = state_0 THEN
57                     write_buf <= data_in;
58                 END IF;
59
60                 IF state = state_trans THEN
61                     read_buf <= y_out_i & read_buf(7 DOWNTO 1);
62                 END IF;
63
64                 state <= next_state;
65                 cnt <= cnt_i;
66             END IF;
67         END IF;
68     END PROCESS seq_p;
69
70     -- combinational process
71     comb_p : PROCESS (state, start, cnt)
72     BEGIN
73         -- default cases
74         next_state <= state;
75         cnt_i <= cnt;
76         y_out_i <= '1';
77
78         CASE state IS
79             WHEN idle =>
80                 y_out_i <= '1';
81                 IF start = '1' THEN
82                     next_state <= state_0;
83                 END IF;
84             WHEN state_0 =>
85                 cnt_i <= 0;
86                 y_out_i <= '0';
87                 next_state <= state_trans;
88             WHEN state_trans =>
89                 y_out_i <= write_buf(cnt);
90                 --read_buf <= y_out_i & read_buf(7 downto 1);
91                 IF cnt = MAXBITS - 1 THEN
92                     next_state <= state_parity;
93                 ELSE
94                     next_state <= state_trans;
95                     cnt_i <= cnt + 1;
96                 END IF;
97             WHEN state_parity =>
98                 y_out_i <= '1' XOR write_buf(MAXBITS - 1) XOR write_buf(MAXBITS - 2)
99                 XOR write_buf(MAXBITS - 3) XOR write_buf(MAXBITS - 4) XOR write_buf(MAXBITS - 5) XOR
100                 write_buf(MAXBITS - 6) XOR write_buf(MAXBITS - 7) XOR write_buf(MAXBITS - 8);
101                 next_state <= state_stop;
102             WHEN state_stop =>

```

```

101         -- stop bit
102         y_out_i <= '1';
103         IF start = '0' THEN
104             next_state <= idle;
105         ELSE
106             next_state <= state_trans;
107         END IF;
108     END CASE;
109 END PROCESS comb_p;
110
111 y_out <= y_out_i;
112 END Behavioral;

```

18 Chirp Signal Generator

The following code shows the implementation of a chirp signal generator in VHDL.

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3  USE ieee.numeric_std.ALL;
4
5  ENTITY s24 IS
6      PORT (
7          clk : IN STD_LOGIC;
8          reset : IN STD_LOGIC;
9          start : IN STD_LOGIC;
10         y_out : OUT STD_LOGIC
11     );
12 END ENTITY;
13
14 ARCHITECTURE behavioral OF s24 IS
15     TYPE state_type IS (idle, state_l, state_h);
16     SIGNAL state, next_state : state_type;
17
18     SUBTYPE clk_cycle_type IS INTEGER RANGE 0 TO 16;
19     TYPE no_of_clk_array IS ARRAY(0 TO 6) OF clk_cycle_type;
20     SIGNAL no_clk_cycles : no_of_clk_array := (7, 6, 5, 4, 3, 1, 0);
21
22     SUBTYPE int16_t IS INTEGER RANGE -32768 TO 32767;
23     SUBTYPE counter_type IS INTEGER RANGE 0 TO 15;
24     SIGNAL timer : counter_type := 0;
25     SIGNAL cnt, cnt_i : int16_t := 0;
26 BEGIN
27
28     -- sequential process
29     -- timed state machine
30     seq_p : PROCESS (clk, reset)
31         VARIABLE count : counter_type := 0;
32     BEGIN
33         IF rising_edge(clk) THEN
34             IF reset = '1' THEN
35                 state <= idle;
36                 count := 0;
37                 cnt <= 0;
38             ELSE
39                 IF count = timer THEN
40                     state <= next_state;
41                     cnt <= cnt_i;
42                     count := 0;
43                 ELSE
44                     count := count + 1;
45                 END IF;
46             END IF;
47         END IF;

```

```

48  END PROCESS;
49
50  -- combinational process
51  comb_p : PROCESS (state, cnt_i, start, cnt)
52  BEGIN
53      -- default cases
54      next_state <= state;
55      cnt_i <= cnt;
56      timer <= 0;
57
58      CASE state IS
59          WHEN idle =>
60              y_out <= '0';
61              cnt_i <= 0;
62              IF start = '1' THEN
63                  next_state <= state_l;
64              END IF;
65          WHEN state_l =>
66              y_out <= '0';
67              timer <= no_clk_cycles(cnt);
68              next_state <= state_h;
69          WHEN state_h =>
70              y_out <= '1';
71              timer <= no_clk_cycles(cnt_i);
72              cnt_i <= cnt + 1;
73
74              IF start = '0' THEN
75                  next_state <= idle;
76                  cnt_i <= 0;
77              ELSE
78                  IF cnt = 6 THEN
79                      cnt_i <= 0;
80                  END IF;
81                  next_state <= state_l;
82              END IF;
83      END CASE;
84  END PROCESS;
85
86  END ARCHITECTURE behavioral;

```

19 Sign Change Counter

The following code shows the implementation of a sign change counter in VHDL.

```

1  LIBRARY ieee;
2  USE ieee.std_logic_1164.ALL;
3  USE ieee.numeric_std.ALL;
4
5  ENTITY s25 IS
6      GENERIC (
7          DATA_NBITS : INTEGER := 10
8      );
9      PORT (
10         clk : IN STD_LOGIC;
11         reset : IN STD_LOGIC;
12         meas_on : IN STD_LOGIC;
13         d_in : IN STD_LOGIC_VECTOR(DATA_NBITS - 1 DOWNTO 0);
14         sign_cnt : OUT STD_LOGIC_VECTOR(DATA_NBITS - 1 DOWNTO 0)
15     );
16  END s25;
17
18  ARCHITECTURE behavioral OF s25 IS
19      TYPE state_type IS (idle, init, sample);
20      SIGNAL state, next_state : state_type;

```

```

21
22     SUBTYPE int16_t IS INTEGER RANGE -32768 TO 32767;
23     SUBTYPE int32_t IS INTEGER RANGE -2147483648 TO 2147483647;
24
25     SIGNAL sign_cnt_i : int16_t := 0;
26     SIGNAL sign_cnt_ii : int16_t := 0;
27     SIGNAL prev_val, prev_val_i, current_val, current_val_i : signed(DATA_NBITS - 1
28 DOWNTO 0) := (OTHERS => '0');
29 BEGIN
30     -- sequential process
31     seq_p : PROCESS (clk)
32     BEGIN
33         IF rising_edge(clk) THEN
34             IF reset = '1' THEN
35                 state <= idle;
36                 sign_cnt_i <= 0;
37                 prev_val <= (OTHERS => '0');
38                 current_val <= (OTHERS => '0');
39             ELSE
40                 state <= next_state;
41                 sign_cnt_i <= sign_cnt_ii;
42                 prev_val <= prev_val_i;
43                 current_val <= current_val_i;
44             END IF;
45         END IF;
46     END PROCESS;
47
48     -- combinational process
49     comb_p : PROCESS (state, meas_on, prev_val, current_val)
50     BEGIN
51         -- default cases
52         next_state <= state;
53         sign_cnt_ii <= sign_cnt_i;
54         prev_val_i <= prev_val;
55         current_val_i <= current_val;
56
57         CASE state IS
58             WHEN idle =>
59                 IF meas_on = '1' THEN
60                     next_state <= init;
61                 END IF;
62             WHEN init =>
63                 prev_val_i <= signed(d_in);
64                 next_state <= sample;
65             WHEN sample =>
66                 IF prev_val_i(DATA_NBITS - 1) /= d_in(DATA_NBITS - 1) THEN
67                     sign_cnt_ii <= sign_cnt_i + 1;
68                 END IF;
69                 IF meas_on = '0' THEN
70                     next_state <= idle;
71                 ELSE
72                     next_state <= sample;
73                 END IF;
74                 prev_val_i <= signed(d_in);
75             END CASE;
76     END PROCESS;
77
78     -- outputs
79     sign_cnt <= STD_LOGIC_VECTOR(to_signed(sign_cnt_ii, sign_cnt'length));
80
81 END ARCHITECTURE behavioral;

```